

WEEK 1

Mobile Application Development

Platforms • Ecosystems • Flutter • Setup

Session 1: Overview & Platforms

Session 2: Flutter Intro

Session 3: SDK Setup

01

SESSION

Overview of Mobile Application Development & Mobile Platforms

CONTINUE

Learning Objectives

- 01** Understand what mobile applications are and how they work in daily life
- 02** Know the major mobile platforms — Android and iOS
- 03** Understand the full mobile app ecosystem and its components
- 04** Recognize real-world examples of mobile apps across different categories

What Are Mobile Applications?

 A mobile application is software program designed to run on smartphones and tablets
Mobile apps solve real-life problems and make daily task easier

Social Media

WhatsApp
Instagram

E-Commerce

Amazon
Daraz

Educational

Duolingo
Corsera

Gaming

PUBG
Subway Surfers

What Is Mobile Application Development?

The process of creating software application for mobile such as smartphone and tablets
Mobile apps solve real-life problems and make daily task easier

Lifecycle:

1. Idea/Problem identification
2. Planning
3. Designing
4. Development
5. Testing
6. Deployment
7. Maintenance

Major Mobile Platforms

A mobile platform is the operating system that runs mobile apps
Just like computer have Windows or macOS

Android

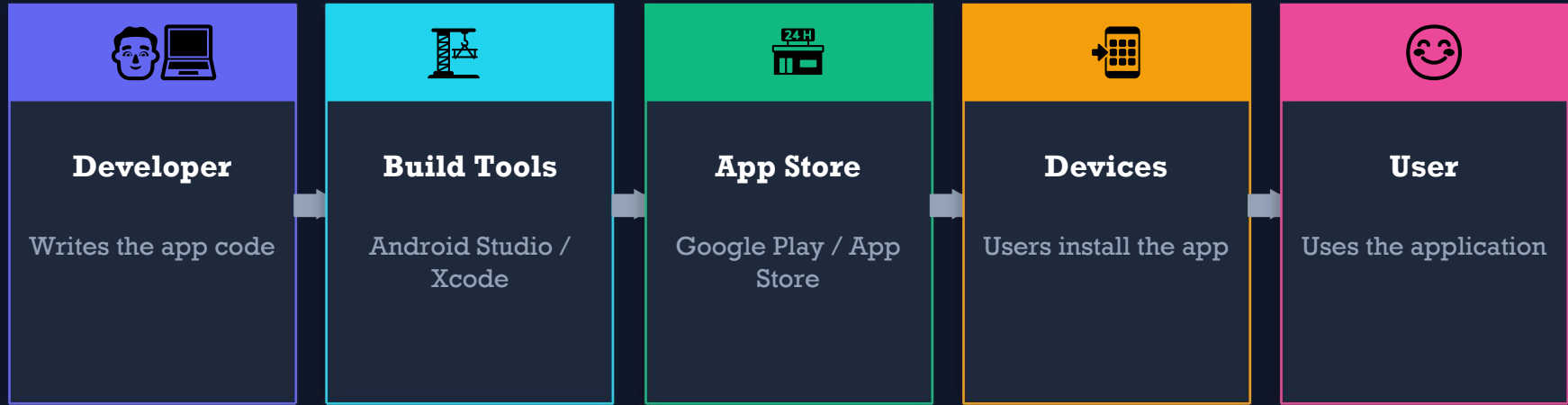
Developer:	Google
Ecosystem:	Open Source
Devices:	Samsung, Xiaomi, Oppo, Vivo...
Language:	Kotlin / Java
Market Share:	~72% Globally

VS

iOS

Developer:	Apple
Ecosystem:	Closed / Controlled
Devices:	iPhone, iPad only
Language:	Swift / Objective-C
Market Share:	~28% Globally

Mobile App Ecosystem



5 Core Components:

Mobile OS

App Store

Dev Tools

Developers

Users

💡 *Key Insight: The ecosystem is a complete chain — from the developer's code editor to the user's fingertip. Every link matters for app success.*

Types of Mobile Applications

⚡ Native Apps

Built specifically for one platform. Offers the best performance and full device access.

✓ Pros:

- Best performance
- Full hardware access
- Smooth UI

⚠ Cons:

- Separate codebase needed
- Higher cost

📦 Android → Kotlin/Java
iOS → Swift/Obj-C

🌐 Web Apps

Run inside a mobile browser. No installation needed — accessible via URL.

✓ Pros:

- No install required
- Cross-platform
- Easy updates

⚠ Cons:

- Limited device access
- Needs internet

📦 HTML • CSS • JavaScript

🔄 Cross-Platform Apps

One codebase, multiple platforms. Balance of speed and reach.

✓ Pros:

- Single codebase
- Faster dev
- Lower cost

⚠ Cons:

- Slight performance gap
- Framework dependency

📦 Flutter • React Native
Xamarin

02

SESSION

Native vs Cross-Platform Development & Flutter Introduction

CONTINUE

Native Development

🔗 Native apps are built specifically for ONE platform using the platform's official language and tools

Android Native

Primary Language:	Kotlin
Legacy Language:	Java
IDE:	Android Studio
Build Tool:	Gradle
UI Toolkit:	Jetpack Compose / XML

iOS Native

Primary Language:	Swift
Legacy Language:	Objective-C
IDE:	Xcode
Build Tool:	Xcode Build
UI Toolkit:	SwiftUI / UIKit

Cross-Platform Development

 **One Codebase**




Android


iOS


Web

Popular Cross-Platform Frameworks:

Flutter

Language: Dart
By: Google
Rating: ★★★★★

React Native

Language: JavaScript
By: Meta
Rating: ★★★★☆

Xamarin

Language: C#
By: Microsoft
Rating: ★★★☆☆



Tradeoff: Cross-platform = faster & cheaper, but native = best performance & device integration

Introduction to Flutter



Flutter

by Google

Language: Dart

Targets:

Android

iOS

Web

Desktop

Hot Reload

See changes instantly without restarting the entire app. Saves massive development time.

Widget-Based

Everything in Flutter is a Widget. Text, Buttons, Images, Layouts — all are widgets.

Beautiful UI

Pixel-perfect, expressive UI with Material Design and Cupertino (iOS-style) components.

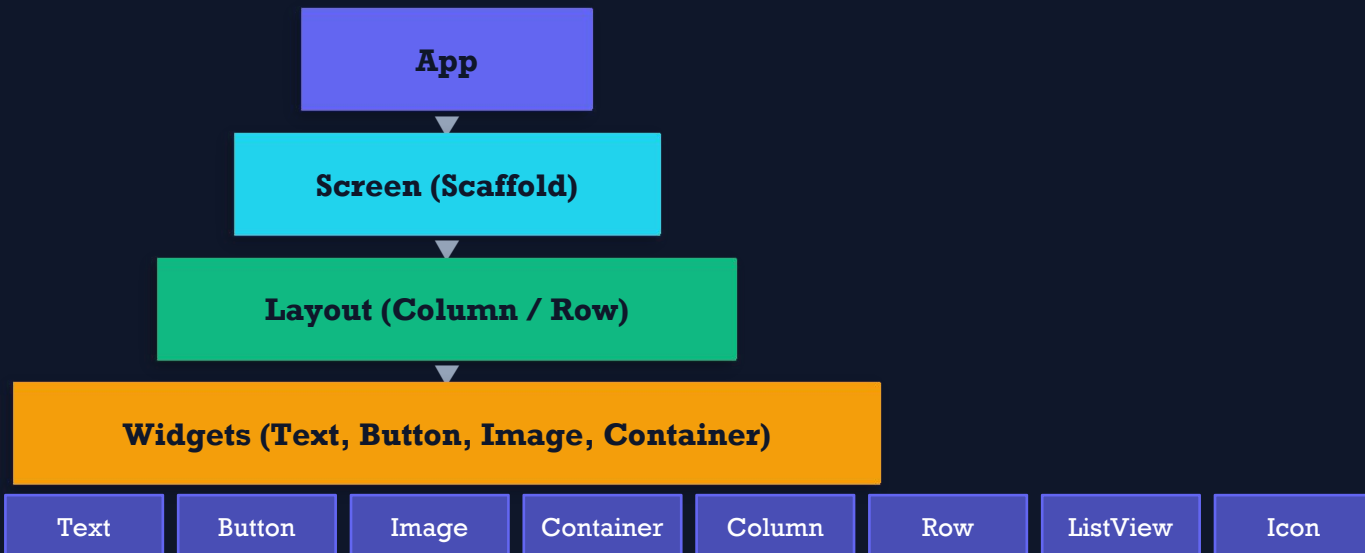
Single Codebase

Write once, deploy to Android, iOS, Web, and Desktop simultaneously.

Flutter Key Concept: Widgets

"Everything in Flutter is a Widget"

Widget Hierarchy:



Common Widgets:

Who Uses Flutter?

Major global companies trust Flutter to build their production apps:



Uses Flutter for internal tools and Google Pay



Xianyu app — 50M+ users built with Flutter



My BMW app for connected car experience



eBay Motors — Flutter-powered marketplace



Multiple apps including NOW Live platform



Award-winning journaling app

 **Flutter is used in 1M+ apps on Google Play and App Store worldwide**

03

SESSION

Installing Flutter SDK & Development Environment Setup

CONTINUE

Flutter Setup Roadmap

1

↓ Download Flutter SDK

Visit flutter.dev → Download for Windows/Mac/Linux → Extract to C:\flutter

2



Add to PATH

Environment Variables → System PATH → Add C:\flutter\bin

3



Install Android Studio

Download Android Studio → Install Android SDK + Emulator → Add Flutter & Dart plugins

4



Run flutter doctor

Open terminal → Run: flutter doctor → Verify ✓ Flutter SDK ✓ Android ✓ VS Code

5



Create First App

flutter create my_first_app → cd my_first_app → flutter run

Understanding: flutter doctor



Terminal

```
$ flutter doctor
```

```
Doctor summary (to see all details, run flutter doctor -v):
```

```
[✓] Flutter (Channel stable, 3.x.x)
```

```
[✓] Android toolchain - develop for Android devices
```

```
[✓] Chrome - develop for the web
```

```
[✓] VS Code (version x.x.x)
```

```
• No issues found!
```

What each check means:

✓ Flutter SDK — Core framework installed and working

✓ Chrome — Web development capability enabled

✓ Android toolchain — Android SDK + ADB + build tools ready

✓ VS Code — Code editor with Flutter extension detected

Creating Your First Flutter App



```
$ flutter create my_first_app  
  
Creating project my_first_app...  
  
$ cd my_first_app  
  
$ flutter run
```

Project Structure

```
my_first_app/  
  lib/  
    main.dart ★  
  android/  
  ios/  
  pubspec.yaml
```

Key File Concepts:

main.dart

Entry point of the app. Contains main() function and Widget tree.

MaterialApp

Top-level widget. Provides Material Design theme, navigation.

Scaffold

Basic screen structure: AppBar, Body, FloatingActionButton.

Hot Reload

Press 'r' in terminal → changes appear instantly without restart!

Week 1 — Complete Summary

Session 1

Overview & Platforms

✓ Mobile apps run on smartphones & tablets

✓ Android (Google) vs iOS (Apple)

✓ Mobile ecosystem: OS → Store → Dev → User

✓ App types: Native, Web, Cross-Platform

Session 2

Flutter Introduction

✓ Native = best perf, one platform

✓ Cross-platform = one codebase, many platforms

✓ Flutter is Google's UI toolkit using Dart

✓ Hot Reload + Widgets are Flutter's superpowers

Session 3

SDK & Environment Setup

✓ Download SDK → Set PATH → Install Android Studio

✓ flutter doctor verifies your setup

✓ flutter create → flutter run = first app!

✓ main.dart → MaterialApp → Scaffold → Widgets